- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 5: Data Visualization using matplotlib

Problem Statement: Analyzing Air Quality Index (AQI) Trends in a City

Prerequisites

pip install pandas numpy matplotlib

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages
(3.10.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (3.3.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0) #Step 0: Generate Sample City_Air_Quality.csv
```

```
import pandas as pd
import numpy as np
import os
```

```
if not os.path.exists("City_Air_Quality.csv"):
np.random.seed(42)
   dates = pd.date_range(start="2026-05-01", periods=180, freq="D")
n = len(dates)

   # Simulate seasonal curve (cleaner air during Monsoon July-Aug, higher pollution in May & Oct)
seasonal_factor = np.cos(np.linspace(0, 2 * np.pi, n))

   pm25 = np.clip(65 + 35 * seasonal_factor + np.random.normal(0, 15, n), 15, 250).round(2)
pm10 = np.clip(pm25 * 1.65 + np.random.normal(15, 20, n), 30, 420).round(2)     no2 =
np.clip(32 + 12 * seasonal_factor + np.random.normal(0, 6, n), 8, 95).round(2)     co =
np.clip(1.1 + 0.45 * seasonal_factor + np.random.normal(0, 0.2, n), 0.2, 3.5).round(2)

   # Calculate composite AQI and assign CPCB AQI Category
   aqi = np.clip(pm25 * 1.45 + pm10 * 0.35 + no2 * 0.25 + np.random.normal(0, 8, n), 35, 450).round(0)

   def get_aqi_bucket(val):
      if val <= 50: return "Good"       elif
val <= 100: return "Satisfactory"       elif
val <= 200: return "Moderate"       elif
val <= 300: return "Poor"       else:
return "Very Poor"

   aqi_category = [get_aqi_bucket(val) for val in aqi]
   dominant_pollutant = np.where(pm25 > 90, "PM2.5", np.where(pm10 > 150, "PM10", "NO2"))

   df_sample = pd.DataFrame({
      "Date": dates.strftime("%Y-%m-%d"),
      "Time": np.random.choice(["08:00:00", "14:00:00", "20:00:00"], n),
      "City": "Pune",
      "PM2.5": pm25,
      "PM10": pm10,
      "NO2": no2,
      "CO": co,
      "AQI": aqi,
      "Dominant_Pollutant": dominant_pollutant,
      "AQI_Category": aqi_category
   })

   df_sample.to_csv("City_Air_Quality.csv",                index=False)
print("Sample  'City_Air_Quality.csv'  generated  successfully.\n")
Sample 'City_Air_Quality.csv' generated successfully.

import  pandas  as  pd  import
numpy      as      np      import
matplotlib.pyplot  as  plt  import
matplotlib.dates as mdates
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.size'] = 10 plt.rcParams['axes.titlesize'] = 12
plt.rcParams['axes.titleweight'] = 'bold'
```

TASK 1 & 2: Import and Explore the Dataset

```
df = pd.read_csv("City_Air_Quality.csv")
print("--- Task 1 & 2: Dataset Structure and Summary ---")
print(f"Dataset Shape: {df.shape}") print("\nData Types &
Non-Null Counts:")
print(df.info())
print("\nDescriptive Statistics:")
print(df.describe().round(2))
print("\nFirst 5 Rows:")
print(df.head())
```

```
--- Task 1 & 2: Dataset Structure and Summary ---
Dataset Shape: (180, 10)

Data Types & Non-Null Counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180 entries, 0 to 179 Data
columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
0   Date                180 non-null    object
1   Time                180 non-null    object
2   City                180 non-null    object
3   PM2.5               180 non-null    float64
4   PM10                180 non-null    float64
5   NO2                 180 non-null    float64
6   CO                  180 non-null    float64
7   AQI                 180 non-null    float64
8   Dominant_Pollutant  180 non-null    object  9   AQI_Category        180 non-null
    object  dtypes: float64(5), object(5) memory usage: 14.2+ KB
None

Descriptive Statistics:
        PM2.5    PM10     NO2      CO     AQI
count  180.00  180.00  180.00  180.00  180.00
mean    65.12  123.11   31.78    1.10  146.62 std
28.48   49.88   10.55    0.38   59.79 min
15.00   30.00    8.01    0.20   35.00 25%
40.79   82.14   23.27    0.83   93.75
50%     63.34  125.40   32.57    1.10  146.50 75%
87.90  161.10   39.90    1.41  192.50 max
140.80  230.78   55.50    1.86  289.00

First 5 Rows:
         Date      Time  City   PM2.5    PM10    NO2    CO    AQI  \
0  2026-05-01  20:00:00  Pune  107.45  204.81  47.12  1.72  242.0
1  2026-05-02  14:00:00  Pune   97.90  159.39  53.19  1.32  197.0
2  2026-05-03  14:00:00  Pune  109.63  174.47  43.32  1.65  220.0
```

```
3  2026-05-04  14:00:00  Pune  122.65  227.02  46.34  1.84  275.0    4  2026-05-05
   08:00:00  Pune   96.14  169.16  48.02  1.05  212.0
```

```
  Dominant_Pollutant AQI_Category
0              PM2.5         Poor
1              PM2.5     Moderate
2              PM2.5         Poor
3              PM2.5         Poor
4              PM2.5         Poor
```

TASK 3: Identify & Prepare Relevant Variables

```
# Convert Date to datetime object and sort chronologically
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date').reset_index(drop=True)

# Extract Month for periodic bar chart comparisons
df['Month'] = df['Date'].dt.to_period('M').astype(str)
df['Month_Name'] = df['Date'].dt.strftime('%b %Y')

# Clean any potential missing numerical values using linear interpolation pollutant_cols
= ['PM2.5', 'PM10', 'NO2', 'CO', 'AQI']
df[pollutant_cols] = df[pollutant_cols].interpolate(method='linear')

# Calculate 7-Day Rolling Average for smoother trend visualization df['AQI_7Day_MA']
= df['AQI'].rolling(window=7, min_periods=1).mean()
```

TASK 4 & 9: Line Plot of Overall AQI Trend Over Time fig,

```
ax = plt.subplots(figsize=(13, 5))

ax.plot(df['Date'], df['AQI'], color='#adb5bd', alpha=0.6, linewidth=1.2, label='Daily AQI')
ax.plot(df['Date'], df['AQI_7Day_MA'], color='#d90429', linewidth=2.5, label='7-Day Moving Average')

# Add horizontal reference lines for AQI safety thresholds ax.axhline(100, color='#2b9348',
linestyle='--', linewidth=1.2, label='Satisfactory Limit (100)') ax.axhline(200, color='#f77f00',
linestyle='--', linewidth=1.2, label='Moderate Limit (200)')

ax.set_title('Task 4: Overall Air Quality Index (AQI) Trend Over Time')
ax.set_xlabel('Date') ax.set_ylabel('AQI Value')
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
ax.legend(loc='upper right', frameon=True) ax.grid(True,
linestyle=':', alpha=0.6)

plt.tight_layout() plt.show()
```



TASK 5 & 9: Separate Line Plots for Individual Pollutants

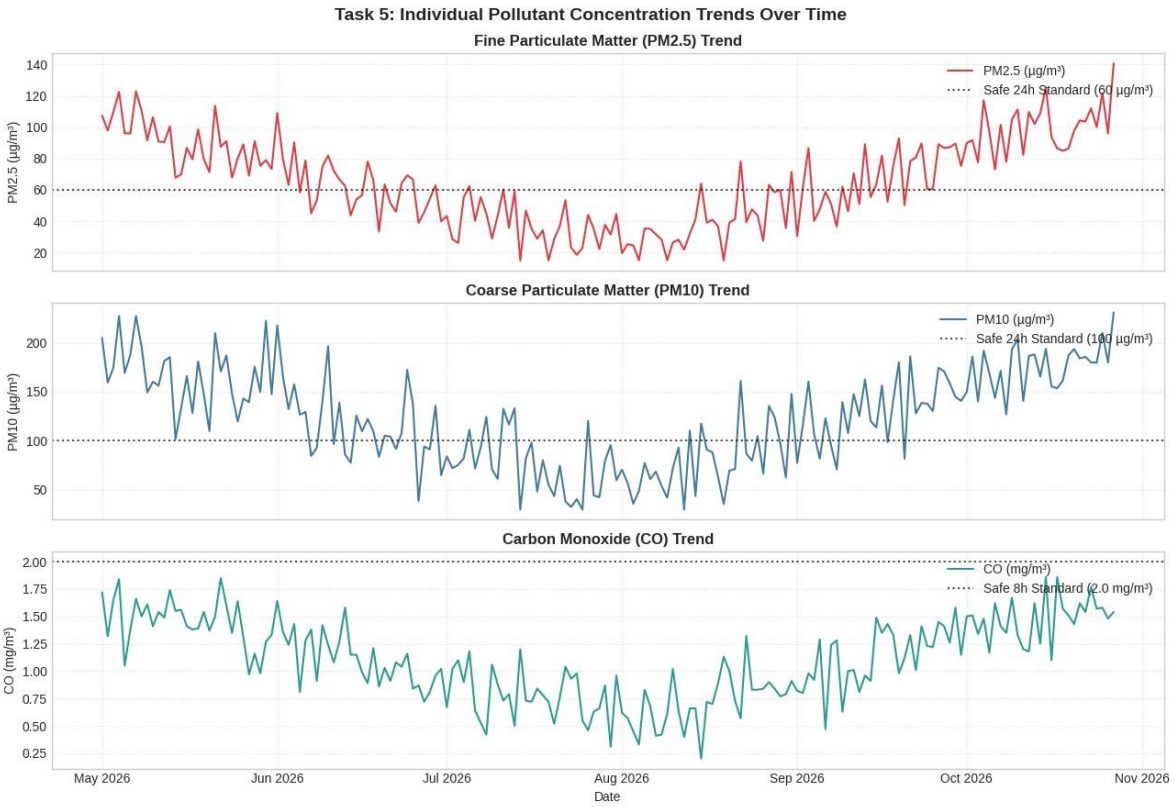
```
fig, axes = plt.subplots(3, 1, figsize=(13, 9), sharex=True)
fig.suptitle('Task 5: Individual Pollutant Concentration Trends Over Time', fontsize=14, fontweight='bold')

# 1. PM2.5 Trend axes[0].plot(df['Date'], df['PM2.5'], color='#e63946', linewidth=1.5,
label='PM2.5 (µg/m³)') axes[0].axhline(60, color='black', linestyle=':', alpha=0.7, label='Safe
24h Standard (60 µg/m³)') axes[0].set_ylabel('PM2.5 (µg/m³)')
axes[0].set_title('Fine Particulate Matter (PM2.5) Trend')
axes[0].legend(loc='upper right') axes[0].grid(True,
linestyle=':', alpha=0.6)

# 2. PM10 Trend
axes[1].plot(df['Date'], df['PM10'], color='#457b9d', linewidth=1.5, label='PM10 (µg/m³)')
axes[1].axhline(100, color='black', linestyle=':', alpha=0.7, label='Safe 24h Standard (100 µg/m³)')
axes[1].set_ylabel('PM10 (µg/m³)')
axes[1].set_title('Coarse Particulate Matter (PM10) Trend')
axes[1].legend(loc='upper right') axes[1].grid(True,
linestyle=':', alpha=0.6)

# 3. Carbon Monoxide (CO) Trend
axes[2].plot(df['Date'], df['CO'], color='#2a9d8f', linewidth=1.5, label='CO (mg/m³)')
axes[2].axhline(2.0, color='black', linestyle=':', alpha=0.7, label='Safe 8h Standard (2.0 mg/m³)')
axes[2].set_ylabel('CO (mg/m³)') axes[2].set_xlabel('Date')
axes[2].set_title('Carbon Monoxide (CO) Trend')
axes[2].xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
axes[2].legend(loc='upper right') axes[2].grid(True, linestyle=':',
alpha=0.6)

plt.tight_layout() plt.show()
```



TASK 6 & 9: Bar Plot & Stacked Bar Plot Across Time Periods

```
# Aggregate monthly average AQI and pollutant levels
monthly_Order = df.groupby('Month', sort=True).agg(
   Month_Label=('Month_Name', 'first'),
   Avg_AQI=('AQI', 'mean'),
   Avg_PM25=('PM2.5', 'mean'),
   Avg_PM10=('PM10', 'mean'),
   Avg_NO2=('NO2', 'mean')
).reset_index()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5.5))

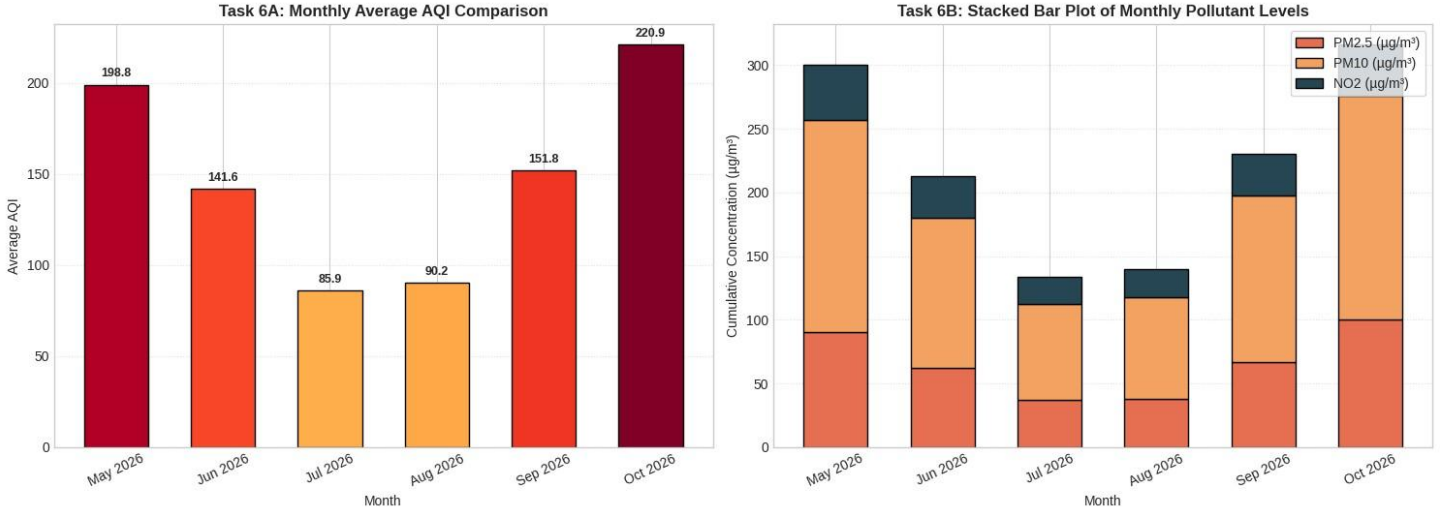
# 6A. Standard Bar Plot: Monthly Average AQI
bars = ax1.bar(
monthly_Order['Month_Label'],
monthly_Order['Avg_AQI'],
   color=plt.cm.YlOrRd(monthly_Order['Avg_AQI'] / monthly_Order['Avg_AQI'].max()),
edgecolor='black',    width=0.6
)
ax1.set_title('Task 6A: Monthly Average AQI Comparison')
ax1.set_xlabel('Month') ax1.set_ylabel('Average AQI')
ax1.tick_params(axis='x', rotation=25) ax1.grid(axis='y',
linestyle=':', alpha=0.6)

# Annotate bar values on top
for bar in bars:      height =
bar.get_height()
   ax1.annotate(f'{height:.1f}', xy=(bar.get_x() + bar.get_width() / 2, height),
            xytext=(0, 4), textcoords="offset points", ha='center', va='bottom', fontsize=9, fontweight='bold')
```

```
# 6B. Stacked Bar Plot: Monthly Pollutant Composition x_labels
= monthly_Order['Month_Label']
p1 = ax2.bar(x_labels, monthly_Order['Avg_PM25'], label='PM2.5 (µg/m³)', color='#e76f51',
edgecolor='black', width=0.6)
p2 = ax2.bar(x_labels, monthly_Order['Avg_PM10'], bottom=monthly_Order['Avg_PM25'], label='PM10
(µg/m³)', color='#f4a261', edgecolor='black', width=0.6)
p3 = ax2.bar(x_labels, monthly_Order['Avg_NO2'], bottom=monthly_Order['Avg_PM25'] +
monthly_Order['Avg_PM10'], label='NO2 (µg/m³)', color='#264653', edgecolor='black', width=0.6)

ax2.set_title('Task 6B: Stacked Bar Plot of Monthly Pollutant Levels') ax2.set_xlabel('Month')
ax2.set_ylabel('Cumulative Concentration (µg/m³)')
ax2.tick_params(axis='x', rotation=25) ax2.legend(loc='upper
right', frameon=True) ax2.grid(axis='y', linestyle=':',
alpha=0.6)

plt.tight_layout() plt.show()
```



TASK 7 & 9: Box Plot & Violin Plot for AQI Distributions

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5.5))

# 7A. Box Plot of AQI grouped by Dominant Pollutant Category
pollutant_groups = ['PM2.5', 'PM10', 'NO2']
box_data = [df[df['Dominant_Pollutant'] == grp]['AQI'].dropna().values for grp in pollutant_groups if grp
in df['Dominant_Pollutant'].values]
valid_labels = [grp for grp in pollutant_groups if grp in df['Dominant_Pollutant'].values]

bp = ax1.boxplot(    box_data,
tick_labels=valid_labels,    patch_artist=True,
notch=False,    boxprops=dict( edgecolor='black',
linewidth=1.2),    medianprops=dict(color='yellow',
linewidth=2)
)
```

```
box_colors = ['#ef476f', '#118ab2', '#06d6a0'] for patch, color in
zip(bp['boxes'], box_colors[:len(valid_labels)]):
patch.set_facecolor(color)    patch.set_alpha(0.8)

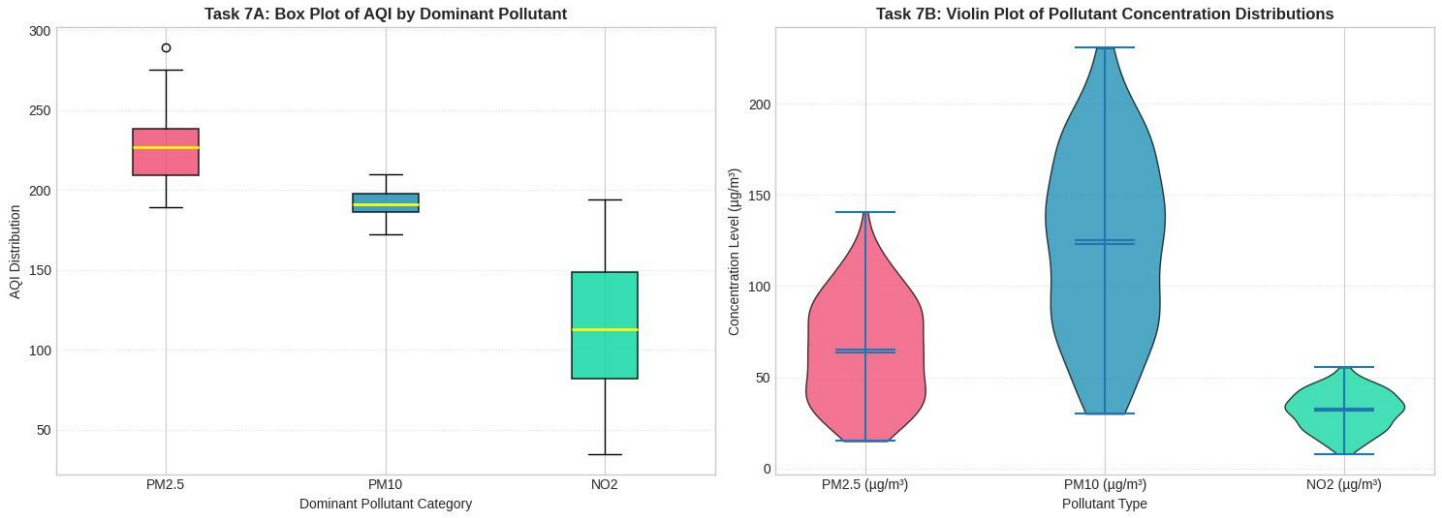
ax1.set_title('Task 7A: Box Plot of AQI by Dominant Pollutant')
ax1.set_xlabel('Dominant Pollutant Category')
ax1.set_ylabel('AQI Distribution') ax1.grid(axis='y',
linestyle=':', alpha=0.6)

# 7B. Violin Plot of Individual Pollutant Concentrations
violin_cols = ['PM2.5', 'PM10', 'NO2']
violin_data = [df[col].dropna().values for col in violin_cols]

vp = ax2.violinplot(violin_data, showmeans=True, showmedians=True) for
i, body in enumerate(vp['bodies']):
  body.set_facecolor(box_colors[i])
body.set_edgecolor('black')    body.set_alpha(0.75)

ax2.set_xticks([1, 2, 3])
ax2.set_xticklabels(['PM2.5 (µg/m³)', 'PM10 (µg/m³)', 'NO2 (µg/m³)'])
ax2.set_title('Task 7B: Violin Plot of Pollutant Concentration Distributions')
ax2.set_xlabel('Pollutant Type') ax2.set_ylabel('Concentration Level
(µg/m³)') ax2.grid(axis='y', linestyle=':', alpha=0.6)

plt.tight_layout() plt.show()
```



TASK 8 & 9: Scatter Plot & Bubble Chart (AQI vs Pollutants)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# 8A. Scatter Plot: PM2.5 vs AQI with Linear Trendline
sc1 = ax1.scatter( df['PM2.5'], df['AQI'],
c=df['AQI'], cmap='plasma', alpha=0.8,
edgecolors='k', linewidth=0.5,
  s=50
)
```

```
# Fit and plot linear regression line m, b =
np.polyfit(df['PM2.5'], df['AQI'], 1)
ax1.plot(df['PM2.5'], m * df['PM2.5'] + b, color='red', linestyle='--', linewidth=2, label=f'Trendline
(Slope={m:.2f})')

cbar1 = plt.colorbar(sc1, ax=ax1)
cbar1.set_label('AQI Intensity') ax1.set_title('Task
8A: Scatter Plot of PM2.5 vs. AQI')
ax1.set_xlabel('PM2.5 Concentration (µg/m³)')
ax1.set_ylabel('Air Quality Index (AQI)')
ax1.legend(loc='upper left')
ax1.grid(True, linestyle=':', alpha=0.6)

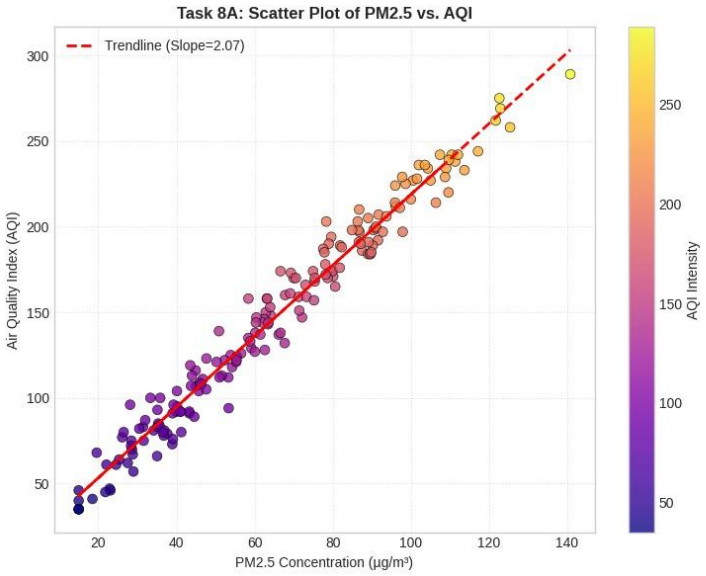
# 8B. Bubble Chart: PM10 vs AQI (Bubble Size = CO Level, Color = PM2.5)
bubble_sizes = df['CO'] * 75  # Scale CO values so bubble radii are visually distinct
sc2 = ax2.scatter(    df['PM10'],    df['AQI'],
   s=bubble_sizes,
c=df['PM2.5'],
cmap='viridis',    alpha=0.75,
edgecolors='black',
linewidth=0.8
)

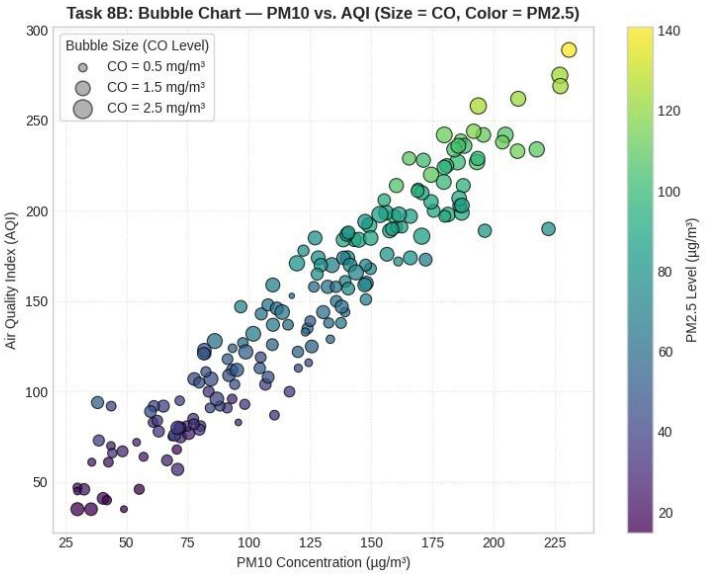

cbar2 = plt.colorbar(sc2, ax=ax2) cbar2.set_label('PM2.5
Level (µg/m³)')

# Create dummy scatter points for bubble size legend for
co_val in [0.5, 1.5, 2.5]:
   ax2.scatter([], [], c='gray', alpha=0.6, s=co_val * 75, edgecolors='black', label=f'CO = {co_val} mg/m³')

ax2.set_title('Task 8B: Bubble Chart — PM10 vs. AQI (Size = CO, Color = PM2.5)')
ax2.set_xlabel('PM10 Concentration (µg/m³)') ax2.set_ylabel('Air Quality Index
(AQI)')
ax2.legend(scatterpoints=1, frameon=True, title='Bubble Size (CO Level)', loc='upper left') ax2.grid(True,
linestyle=':', alpha=0.6)

plt.tight_layout() plt.show()
```


Task 8A: Scatter Plot of PM2.5 vs. AQI
Trendline (Slope=2.07)
Air Quality Index (AQI)
PM2.5 Concentration (µg/m³)
AQI Intensity


Task 8B: Bubble Chart — PM10 vs. AQI (Size = CO, Color = PM2.5)
Bubble Size (CO Level)
CO = 0.5 mg/m³
CO = 1.5 mg/m³
CO = 2.5 mg/m³
Air Quality Index (AQI)
PM10 Concentration (µg/m³)
PM2.5 Level (µg/m³)